Software Processes: Software Lifecycle and Process Models

Software Process: Set of activities required to develop software. Software processes ensure that development is not chaotic but guided, predictable, and efficient.

Activities involved are:

1. Requirements analysis
2. Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

Lifecycle Models:

1. Waterfall Model: Sequential, rigid.
2. Incremental Model: Builds software in small pieces.
3. Spiral Model: Iterative with risk analysis.
4. Agile: Adaptive, customer-focused, iterative sprints.

Software does not emerge from random coding activities. It is the result of an organized set of steps known as the software process. A software process defines how software is planned, built, delivered, and maintained. It guides engineers from the earliest idea through to the final product and its eventual retirement.

A structured process is crucial because software systems today are complex, often mission-critical, and highly interdependent. Without a process, projects face the risks of cost overruns, missed deadlines, and system failures.

The software process models – the different structured approaches to organizing these stages.

Software Lifecycle
Meaning of Software Lifecycle

The software lifecycle refers to the stages a software product passes through from its initial conception until it is retired from use. These stages reflect the natural “life” of software, similar to how physical products have a design, usage, and eventual disposal phase.
The software lifecycle – the sequence of stages every software system undergoes. The software lifecycle outlines the stages software undergoes from conception to retirement. The lifecycle shows that software is not static. It evolves continuously, demanding ongoing attention from engineers.

Major Phases of the Software Lifecycle

1. Requirements Analysis: involves gathering and documenting user needs. It ensures engineers understand what the system should do.

Example: For a university portal, requirements may include online registration, grade viewing, and fee payment.

2. System and Software Design: it involves translating requirements into architecture, modules, and interfaces. It provides the blueprint for implementation.

3. Implementation (Coding): this is actual development of software components in chosen programming languages. It must adhere to coding standards and best practices.

4. Testing and Verification: Ensures each component and the whole system function correctly. It covers unit, integration, system, and acceptance testing.

5. Deployment: Releasing the software to the users. It may include installation, training, and data migration.

6. Maintenance and Evolution: Bug fixing, performance tuning, and updating features. Often the longest and most costly phase, since software must evolve as user needs change.

Software Process Models
What Are Process Models?
A process model provides a structured framework/ approaches for carrying out the lifecycle stages. While the lifecycle tells us what stages exist, the process model tells us how these stages are organized, executed, and related. Different projects require different models depending on size, complexity, and risk.

Common Software Process Models
1. Waterfall Model: Sequential flow from requirements → design → implementation → testing → deployment → maintenance.

Strengths:
Clear documentation.
Easy to manage for small, well-defined projects.

Weaknesses:
Inflexible; changes are difficult once a stage is completed.
Testing comes late, so major defects may surface late.

Example: Military or government projects where requirements are rigid and rarely change.

2. Iterative Model: Software is developed in small portions; each iteration goes through requirements, design, coding, and testing.

Strengths:
Early feedback from users.
Easier to handle changes.

Weaknesses:
Requires careful management to avoid scope creep.

Example: Banking software that evolves with new regulatory requirements.

3. Spiral Model: Combines iterative development with risk analysis. The process “spirals” through repeated cycles of planning, risk assessment, development, and evaluation.

Strengths:
Explicitly focuses on risk management.
Useful for large, high-risk projects.

Weaknesses:
Complex to manage.
Costly for small projects.

Example: Aerospace or defense software where safety is critical.

4. Agile Model: Emphasizes flexibility, customer collaboration, and iterative delivery. Work is done in short cycles (sprints), typically lasting 1–4 weeks.

Strengths:
Highly adaptive to changing requirements.
Continuous feedback ensures higher customer satisfaction.

Weaknesses:
Requires strong customer involvement.
May struggle with very large or distributed teams.

Example: Startups building web applications that evolve rapidly with user feedback.

5. V-Model (Verification and Validation Model)

Structure: An extension of the waterfall model, where each development stage has a corresponding testing activity (e.g., requirements ↔ acceptance testing).

Strengths:
Emphasizes testing at every stage.
Useful for safety-critical systems.

Weaknesses:
Still rigid, like the waterfall model.

Example: Medical device software where validation is as important as development.

Factors to consider when selecting a process model are:

1. Project size and complexity (large projects may prefer Spiral or V-Model).
2. Requirement stability (stable → Waterfall, unstable → Agile).
3. Risk level (high risk → Spiral, low risk → Iterative or Agile).
4. Time-to-market pressure (Agile is best suited for rapid releases).

Relationship Between Lifecycle and Process Models
The software lifecycle is universal; every software passes through requirements, design, coding, testing, deployment, and maintenance. The process model, however, determines how these stages are organized and managed. For example:

In the Waterfall model, testing comes only after development.
In Agile, testing is continuous throughout development.
In the Spiral model, risk analysis is an additional recurring activity.
Thus, lifecycle is the what, process model is the how.

A good engineer understands that there is no single "best" model. Instead, the model must fit the project's context—its size, risks, goals, and stakeholders. In professional practice, mastering lifecycle phases and process models helps not only to build software but also to manage complexity, reduce risks, and deliver quality systems in real-world environments.

Scope creep

Scope creep is the uncontrolled expansion of a project's scope—meaning new features, requirements, or changes are added to a project after it has started, without proper review, planning, or adjustment of time, cost, and resources.

Example:

A client asks for a mobile app that tracks expenses. Initially, the agreed features include expense input, reports, and charts. Midway, the client suddenly requests: Integration with multiple bank APIs. AI-based spending predictions. A social sharing feature. If these are added without adjusting the budget, schedule, or resources, that's scope creep.

Causes of Scope Creep
Unclear requirements at the start of the project.
Poor communication between stakeholders and developers.
Lack of formal change management procedures.
Over-eagerness to please clients, leading to uncontrolled acceptance of requests.

Consequences of Scope Creep
Project delays because of unplanned work.
Budget overruns due to additional features.
Lower product quality, as teams rush to fit everything in.
Team frustration and burnout.
Sometimes project failure, if expansion becomes unmanageable.

How to Manage Scope Creep
Clearly define and document requirements before starting.
Use a formal change request process for new features.
Apply Agile methodologies, where changes are expected but controlled within iterations.
Communicate openly with stakeholders about time and cost implications of new requests.

Scope creep is like promising to bake a simple cake, but while you're baking, people keep asking you to add more layers, new flavors, and decorations—without giving you more time or ingredients.

Scope Change

Definition: A formal, controlled modification of the project's scope. It is documented, reviewed, and approved through a structured change management process. It can add value while still managing risk, time, and budget.

Example:

A stakeholder requests an extra reporting feature. The project team analyzes its impact, adjusts the schedule and cost, and formally approves it before proceeding.